



# Complete Software Engineer Roadmap - Entry Level Jobs



## Overview

Yeh roadmap specially design kiya gaya hai un students ke liye jo software engineering mein entry-level job dhundh rahe hain. Isme comprehensive coverage hai DSA, Projects, System Design, Core CS Fundamentals aur Programming Languages ka.

---

## 1 DATA STRUCTURES & ALGORITHMS (DSA)

**Target: 75 Patterns × 6 Questions = 450 Questions**

### Pattern 1-10: Arrays & Strings

1. **Two Pointers Pattern** (6 Questions)
  - Container With Most Water
  - 3Sum Problem
  - Remove Duplicates from Sorted Array
  - Valid Palindrome
  - Trapping Rain Water
  - Sort Colors (Dutch National Flag)
2. **Sliding Window Pattern** (6 Questions)
  - Maximum Sum Subarray of Size K
  - Longest Substring Without Repeating Characters
  - Minimum Window Substring
  - Permutation in String
  - Longest Repeating Character Replacement
  - Maximum Consecutive Ones III
3. **Prefix Sum Pattern** (6 Questions)
  - Range Sum Query - Immutable
  - Subarray Sum Equals K
  - Continuous Subarray Sum
  - Product of Array Except Self
  - Find Pivot Index
  - Maximum Size Subarray Sum Equals k
4. **Kadane's Algorithm Pattern** (6 Questions)

- Maximum Subarray Sum
- Maximum Product Subarray
- Maximum Sum Circular Subarray
- Longest Turbulent Subarray
- Best Time to Buy and Sell Stock
- Maximum Absolute Sum of Any Subarray

**5. String Manipulation Pattern (6 Questions)**

- Reverse Words in a String
- Implement strStr()
- Longest Common Prefix
- Group Anagrams
- Valid Anagram
- Isomorphic Strings

**6. Array Rotation Pattern (6 Questions)**

- Rotate Array
- Search in Rotated Sorted Array
- Find Minimum in Rotated Sorted Array
- Rotate Image
- Spiral Matrix
- Set Matrix Zeroes

**7. Merge Intervals Pattern (6 Questions)**

- Merge Intervals
- Insert Interval
- Non-overlapping Intervals
- Meeting Rooms II
- Interval List Intersections
- Employee Free Time

**8. Matrix Traversal Pattern (6 Questions)**

- Spiral Matrix Traversal
- Search a 2D Matrix
- Valid Sudoku
- Number of Islands
- Rotate Image 90 Degrees
- Word Search

**9. Dutch National Flag Pattern (6 Questions)**

- Sort Colors
- Sort Array By Parity
- Partition Array According to Given Pivot
- Wiggle Sort
- Move Zeroes
- Separate Even and Odd Numbers

**10. Fast & Slow Pointers (Array) (6 Questions)**

- Find Duplicate Number
- Happy Number
- Circular Array Loop
- Find the Middle of Linked List
- Palindrome Linked List
- Remove Nth Node From End

## **Pattern 11-20: Linked Lists & Stacks/Queues**

### **11. Linked List Reversal Pattern** (6 Questions)

- Reverse Linked List
- Reverse Linked List II
- Reverse Nodes in k-Group
- Swap Nodes in Pairs
- Palindrome Linked List
- Add Two Numbers

### **12. Linked List Cycle Pattern** (6 Questions)

- Linked List Cycle
- Linked List Cycle II
- Happy Number
- Find Duplicate Number
- Circular Linked List
- Intersection of Two Linked Lists

### **13. Merge Lists Pattern** (6 Questions)

- Merge Two Sorted Lists
- Merge K Sorted Lists
- Sort List
- Intersection of Two Linked Lists
- Add Two Numbers II
- Flatten Multilevel Doubly Linked List

### **14. Stack Basic Operations** (6 Questions)

- Valid Parentheses
- Min Stack
- Implement Queue using Stacks
- Implement Stack using Queues
- Baseball Game
- Next Greater Element I

### **15. Monotonic Stack Pattern** (6 Questions)

- Next Greater Element II
- Daily Temperatures
- Largest Rectangle in Histogram
- Maximal Rectangle
- Trapping Rain Water

- Remove K Digits

**16. Queue Pattern** (6 Questions)

- Design Circular Queue
- Sliding Window Maximum
- Number of Recent Calls
- Design Hit Counter
- Moving Average from Data Stream
- Dota2 Senate

**17. Expression Evaluation Pattern** (6 Questions)

- Basic Calculator
- Basic Calculator II
- Evaluate Reverse Polish Notation
- Decode String
- Remove All Adjacent Duplicates In String
- Simplify Path

**18. Histogram Pattern** (6 Questions)

- Largest Rectangle in Histogram
- Maximal Rectangle
- Maximum Width Ramp
- Sum of Subarray Minimums
- Online Stock Span
- Car Fleet

**19. Parentheses Pattern** (6 Questions)

- Generate Parentheses
- Remove Invalid Parentheses
- Longest Valid Parentheses
- Different Ways to Add Parentheses
- Minimum Add to Make Parentheses Valid
- Score of Parentheses

**20. Deque Pattern** (6 Questions)

- Sliding Window Maximum
- Shortest Subarray with Sum at Least K
- Jump Game VI
- Constrained Subsequence Sum
- Maximum of All Subarrays of Size K
- Design Circular Deque

**Pattern 21-30: Trees & Graphs**

**21. Tree Traversal Pattern** (6 Questions)

- Binary Tree Inorder Traversal
- Binary Tree Preorder Traversal

- Binary Tree Postorder Traversal
- Level Order Traversal
- Zigzag Level Order Traversal
- Vertical Order Traversal

**22. Tree DFS Pattern** (6 Questions)

- Maximum Depth of Binary Tree
- Path Sum
- Path Sum II
- Diameter of Binary Tree
- Binary Tree Maximum Path Sum
- Lowest Common Ancestor

**23. Tree BFS Pattern** (6 Questions)

- Binary Tree Level Order Traversal
- Binary Tree Right Side View
- Average of Levels in Binary Tree
- Minimum Depth of Binary Tree
- Populating Next Right Pointers
- Binary Tree Zigzag Level Order

**24. Binary Search Tree Pattern** (6 Questions)

- Validate Binary Search Tree
- Kth Smallest Element in BST
- Lowest Common Ancestor of BST
- Convert Sorted Array to BST
- Delete Node in a BST
- Inorder Successor in BST

**25. Tree Construction Pattern** (6 Questions)

- Construct Binary Tree from Preorder and Inorder
- Construct Binary Tree from Inorder and Postorder
- Serialize and Deserialize Binary Tree
- Clone Graph
- Copy List with Random Pointer
- Flatten Binary Tree to Linked List

**26. Graph DFS Pattern** (6 Questions)

- Number of Islands
- Clone Graph
- Pacific Atlantic Water Flow
- Course Schedule
- Course Schedule II
- All Paths From Source to Target

**27. Graph BFS Pattern** (6 Questions)

- Word Ladder
- Shortest Path in Binary Matrix

- Rotting Oranges
- Open the Lock
- Minimum Knight Moves
- Snakes and Ladders

**28. Union Find Pattern** (6 Questions)

- Number of Connected Components
- Graph Valid Tree
- Redundant Connection
- Accounts Merge
- Satisfiability of Equality Equations
- Smallest String With Swaps

**29. Topological Sort Pattern** (6 Questions)

- Course Schedule
- Course Schedule II
- Alien Dictionary
- Sequence Reconstruction
- Minimum Height Trees
- Sort Items by Groups Respecting Dependencies

**30. Trie Pattern** (6 Questions)

- Implement Trie
- Add and Search Word
- Word Search II
- Replace Words
- Longest Word in Dictionary
- Design Search Autocomplete System

**Pattern 31-40: Recursion & Backtracking**

**31. Subset Pattern** (6 Questions)

- Subsets
- Subsets II
- Permutations
- Permutations II
- Combinations
- Combination Sum

**32. Permutation Pattern** (6 Questions)

- Permutations
- Permutations II
- Next Permutation
- Letter Case Permutation
- Beautiful Arrangement
- String Permutation

**33. Combination Pattern** (6 Questions)

- Combination Sum
- Combination Sum II
- Combination Sum III
- Letter Combinations of Phone Number
- Generate Parentheses
- Factor Combinations

**34. Partition Pattern** (6 Questions)

- Palindrome Partitioning
- Partition to K Equal Sum Subsets
- Word Break II
- Restore IP Addresses
- Decode Ways II
- Split Array into Fibonacci Sequence

**35. N-Queens Pattern** (6 Questions)

- N-Queens
- N-Queens II
- Sudoku Solver
- Valid Sudoku
- Word Search
- Android Unlock Patterns

**36. Word Search Pattern** (6 Questions)

- Word Search
- Word Search II
- Exist Word in Grid
- Boggle Game
- Word Ladder
- Word Pattern Matching

**37. Tree Recursion Pattern** (6 Questions)

- Balanced Binary Tree
- Same Tree
- Symmetric Tree
- Invert Binary Tree
- Subtree of Another Tree
- Merge Two Binary Trees

**38. Divide and Conquer Pattern** (6 Questions)

- Merge Sort
- Quick Sort
- Pow(x, n)
- Kth Largest Element
- Maximum Subarray
- Different Ways to Add Parentheses

### **39. Memoization Pattern (6 Questions)**

- Fibonacci Number
- Climbing Stairs
- Decode Ways
- Unique Paths
- Coin Change
- Word Break

### **40. Matrix Backtracking Pattern (6 Questions)**

- Rat in a Maze
- Knight's Tour
- Sudoku Solver
- N-Queens
- Word Search
- Unique Paths III

## **Pattern 41-50: Dynamic Programming**

### **41. Linear DP Pattern (6 Questions)**

- Climbing Stairs
- House Robber
- House Robber II
- Decode Ways
- Coin Change
- Perfect Squares

### **42. Knapsack Pattern (0/1) (6 Questions)**

- 0/1 Knapsack
- Partition Equal Subset Sum
- Target Sum
- Last Stone Weight II
- Ones and Zeroes
- Profitable Schemes

### **43. Unbounded Knapsack Pattern (6 Questions)**

- Unbounded Knapsack
- Coin Change
- Coin Change 2
- Minimum Cost For Tickets
- Perfect Squares
- Integer Break

### **44. Fibonacci DP Pattern (6 Questions)**

- Fibonacci Number
- Climbing Stairs
- Min Cost Climbing Stairs



- Tribonacci Number
- N-th Tribonacci Number
- House Robber

**45. Longest Common Subsequence Pattern (6 Questions)**

- Longest Common Subsequence
- Longest Common Substring
- Edit Distance
- Delete Operation for Two Strings
- Minimum ASCII Delete Sum
- Shortest Common Supersequence

**46. Longest Increasing Subsequence Pattern (6 Questions)**

- Longest Increasing Subsequence
- Number of LIS
- Russian Doll Envelopes
- Maximum Length of Pair Chain
- Largest Divisible Subset
- Increasing Triplet Subsequence

**47. Palindrome DP Pattern (6 Questions)**

- Longest Palindromic Substring
- Longest Palindromic Subsequence
- Palindrome Partitioning II
- Count Palindromic Substrings
- Palindromic Substrings
- Minimum Insertion Steps to Make String Palindrome

**48. Matrix DP Pattern (6 Questions)**

- Unique Paths
- Unique Paths II
- Minimum Path Sum
- Maximum Square
- Dungeon Game
- Cherry Pickup

**49. DP on Strings Pattern (6 Questions)**

- Longest Common Subsequence
- Edit Distance
- Distinct Subsequences
- Regular Expression Matching
- Wildcard Matching
- Interleaving String

**50. State Machine DP Pattern (6 Questions)**

- Best Time to Buy and Sell Stock
- Best Time to Buy and Sell Stock II
- Best Time to Buy and Sell Stock III

- Best Time to Buy and Sell Stock IV
- Best Time to Buy and Sell Stock with Cooldown
- Best Time to Buy and Sell Stock with Transaction Fee

## **Pattern 51-60: Greedy & Advanced Patterns**

### **51. Greedy Interval Pattern (6 Questions)**

- Jump Game
- Jump Game II
- Gas Station
- Candy
- Task Scheduler
- Non-overlapping Intervals

### **52. Greedy Array Pattern (6 Questions)**

- Maximum Subarray
- Container With Most Water
- Wiggle Subsequence
- Partition Labels
- Reorganize String
- Queue Reconstruction by Height

### **53. Two Heaps Pattern (6 Questions)**

- Find Median from Data Stream
- Sliding Window Median
- IPO
- Find Right Interval
- Schedule Tasks
- Maximize Capital

### **54. K-way Merge Pattern (6 Questions)**

- Merge K Sorted Lists
- Kth Smallest Element in Sorted Matrix
- Smallest Range Covering Elements
- Find K Pairs with Smallest Sums
- Merge K Sorted Arrays
- Kth Smallest Prime Fraction

### **55. Top K Elements Pattern (6 Questions)**

- Kth Largest Element in Array
- Top K Frequent Elements
- Kth Largest Element in Stream
- K Closest Points to Origin
- Top K Frequent Words
- Reorganize String

### **56. Modified Binary Search Pattern (6 Questions)**

- Search in Rotated Sorted Array
- Find Minimum in Rotated Sorted Array
- Search in 2D Matrix
- Find Peak Element
- Search Insert Position
- First Bad Version

#### 57. **Binary Search on Answer Pattern** (6 Questions)

- Koko Eating Bananas
- Minimum Number of Days to Make m Bouquets
- Capacity To Ship Packages Within D Days
- Split Array Largest Sum
- Find K-th Smallest Pair Distance
- Magnetic Force Between Two Balls

#### 58. **Bit Manipulation Pattern** (6 Questions)

- Single Number
- Single Number II
- Single Number III
- Number of 1 Bits
- Counting Bits
- Reverse Bits

#### 59. **XOR Pattern** (6 Questions)

- Find Missing Number
- Find Two Missing Numbers
- Complement of Base 10 Integer
- Flip Image
- Single Number Variants
- Maximum XOR of Two Numbers

#### 60. **Cyclic Sort Pattern** (6 Questions)

- Missing Number
- Find All Duplicates in Array
- Find All Numbers Disappeared
- Find Duplicate Number
- First Missing Positive
- Find Corrupt Pair

### **Pattern 61-70: Advanced Data Structures**

#### 61. **Segment Tree Pattern** (6 Questions)

- Range Sum Query - Mutable
- Range Minimum Query
- Count of Smaller Numbers After Self
- Range Sum Query 2D - Mutable
- The Skyline Problem

- Falling Squares

**62. Binary Indexed Tree Pattern** (6 Questions)

- Range Sum Query - Mutable
- Count of Smaller Numbers After Self
- Reverse Pairs
- Count of Range Sum
- Rectangle Area II
- Number of Longest Increasing Subsequence

**63. Disjoint Set Union Pattern** (6 Questions)

- Friend Circles
- Number of Provinces
- Redundant Connection
- Most Stones Removed
- Accounts Merge
- Satisfiability of Equality Equations

**64. Hashing Pattern** (6 Questions)

- Two Sum
- Group Anagrams
- Isomorphic Strings
- Word Pattern
- Valid Sudoku
- Contains Duplicate II

**65. HashMap Design Pattern** (6 Questions)

- Design HashMap
- Design HashSet
- LRU Cache
- LFU Cache
- Insert Delete GetRandom O(1)
- Time Based Key-Value Store

**66. Priority Queue Pattern** (6 Questions)

- Kth Largest Element
- Merge K Sorted Lists
- Find Median from Data Stream
- Task Scheduler
- Reorganize String
- Rearrange String k Distance Apart

**67. Design Pattern** (6 Questions)

- LRU Cache
- LFU Cache
- Design Twitter
- Design Tic-Tac-Toe
- Design Snake Game

- Design Search Autocomplete System

**68. Rolling Hash Pattern** (6 Questions)

- Repeated DNA Sequences
- Longest Duplicate Substring
- Shortest Palindrome
- Distinct Echo Substrings
- Find All Good Strings
- Rabin-Karp String Matching

**69. Suffix Array Pattern** (6 Questions)

- Longest Repeated Substring
- Longest Common Substring
- Count Distinct Substrings
- Burrows-Wheeler Transform
- Longest Duplicate Substring
- Substring with Concatenation

**70. String Hashing Pattern** (6 Questions)

- Repeated DNA Sequences
- Distinct Echo Substrings
- Longest Duplicate Substring
- Find All Good Strings
- Rolling Hash Implementation
- Palindrome Pairs

**Pattern 71-75: Specialized Patterns**

**71. Game Theory Pattern** (6 Questions)

- Nim Game
- Stone Game
- Stone Game II
- Predict the Winner
- Can I Win
- Flip Game II

**72. Math Pattern** (6 Questions)

- Pow(x, n)
- Sqrt(x)
- Integer to Roman
- Roman to Integer
- Factorial Trailing Zeroes
- Excel Sheet Column Number

**73. Geometry Pattern** (6 Questions)

- Rectangle Area
- Rectangle Overlap

- Valid Square
- Minimum Area Rectangle
- Max Points on a Line
- Convex Polygon

#### 74. **Probability & Randomization** (6 Questions)

- Random Pick Index
- Random Pick with Weight
- Shuffle an Array
- Linked List Random Node
- Random Point in Non-overlapping Rectangles
- Generate Random Point in Circle

#### 75. **Simulation Pattern** (6 Questions)

- Robot Bounded In Circle
- Walking Robot Simulation
- Spiral Matrix
- Design Tic-Tac-Toe
- Design Snake Game
- Design Parking System

## **2 PROJECTS - 7:5:3:2 GOLDEN RATIO**

**Total: 17 Projects (7 Easy + 5 Medium + 3 Hard + 2 Research-Based)**

### **7 EASY PROJECTS (Learning Basics)**

#### **Project 1: Personal Portfolio Website**

**Tech Stack:** HTML, CSS, JavaScript, Responsive Design

- **Features:**
  - Responsive landing page with hero section
  - About me section with skills showcase
  - Projects gallery with hover effects
  - Contact form with validation
  - Smooth scrolling and animations
  - Mobile-first design approach
- **Learning Outcomes:**
  - HTML5 semantic elements
  - CSS Flexbox and Grid
  - JavaScript DOM manipulation
  - Form validation
  - Responsive design principles
  - Git version control basics
- **Duration:** 1 week

## Project 2: Todo List Application

**Tech Stack:** React.js, LocalStorage, CSS Modules

- **Features:**
  - Add, edit, delete tasks
  - Mark tasks as complete
  - Filter tasks (All, Active, Completed)
  - LocalStorage for data persistence
  - Task categories/tags
  - Due date functionality
- **Learning Outcomes:**
  - React components and props
  - State management with hooks
  - Event handling in React
  - Browser storage APIs
  - Component lifecycle
  - Conditional rendering
- **Duration:** 1 week

## Project 3: Weather App

**Tech Stack:** JavaScript, REST API, OpenWeatherMap API

- **Features:**
  - Search city weather
  - Display current temperature, humidity, wind speed
  - 5-day forecast
  - Geolocation support
  - Weather icons based on conditions
  - Unit conversion (Celsius/Fahrenheit)
- **Learning Outcomes:**
  - API integration and fetch
  - Async/await and Promises
  - JSON data handling
  - Error handling
  - Geolocation API
  - Environment variables
- **Duration:** 1 week

## Project 4: Expense Tracker

**Tech Stack:** React, Chart.js, Context API

- **Features:**
  - Add income and expenses
  - Categorize transactions
  - Visual charts for spending analysis
  - Monthly/yearly summaries

- Export data to CSV
- Budget limits with alerts
- **Learning Outcomes:**
  - React Context API
  - Data visualization
  - State management patterns
  - Date handling
  - File download functionality
  - Custom hooks
- **Duration:** 1.5 weeks

## **Project 5: Blog Platform (Frontend)**

**Tech Stack:** React, React Router, Markdown

- **Features:**
  - List of blog posts
  - Individual post pages
  - Markdown support for writing
  - Search and filter functionality
  - Categories and tags
  - Reading time estimation
- **Learning Outcomes:**
  - React Router navigation
  - Markdown parsing
  - Search algorithms
  - Pagination
  - URL parameters
  - Code splitting
- **Duration:** 1.5 weeks

## **Project 6: Quiz Application**

**Tech Stack:** JavaScript, HTML/CSS, LocalStorage

- **Features:**
  - Multiple choice questions
  - Timer for each question
  - Score calculation
  - Result summary with correct answers
  - Different quiz categories
  - Leaderboard
- **Learning Outcomes:**
  - Timer implementation
  - Complex state management
  - Array methods and manipulation
  - LocalStorage for scores
  - Animation and transitions
  - Progressive web app basics



- **Duration:** 1 week

### **Project 7: Recipe Finder**

**Tech Stack:** React, Spoonacular API, CSS-in-JS (Styled Components)

- **Features:**
    - Search recipes by ingredients
    - Display recipe details and instructions
    - Save favorite recipes
    - Dietary filters (vegan, vegetarian, etc.)
    - Nutritional information
    - Shopping list generator
  - **Learning Outcomes:**
    - Complex API integration
    - Styled Components
    - Advanced filtering
    - Favorites management
    - Grid layouts
    - Debouncing search
  - **Duration:** 1.5 weeks
- 

## **5 MEDIUM PROJECTS (Full-Stack Development)**

### **Project 8: E-Commerce Store**

**Tech Stack:** MERN (MongoDB, Express, React, Node.js)

- **Features:**
  - User authentication (signup/login)
  - Product listing with search and filters
  - Shopping cart functionality
  - Checkout process
  - Order history
  - Admin panel for product management
  - Payment integration (Stripe/Razorpay)
  - Email notifications
- **Learning Outcomes:**
  - Full-stack development
  - JWT authentication
  - RESTful API design
  - MongoDB schema design
  - Payment gateway integration
  - File upload (product images)
  - Email services
  - Security best practices
- **Duration:** 3-4 weeks

## **Project 9: Real-Time Chat Application**

**Tech Stack:** Node.js, Socket.io, React, MongoDB

- **Features:**
  - Real-time messaging
  - Multiple chat rooms
  - User presence (online/offline)
  - Typing indicators
  - Message history
  - File sharing
  - Emoji support
  - Notifications
- **Learning Outcomes:**
  - WebSocket protocol
  - Socket.io implementation
  - Real-time event handling
  - Chat room architecture
  - File upload and storage
  - Notification system
  - Optimistic UI updates
- **Duration:** 3 weeks

## **Project 10: Task Management System (Trello Clone)**

**Tech Stack:** React, Node.js, PostgreSQL, Drag-and-Drop

- **Features:**
  - Multiple boards and lists
  - Drag-and-drop cards
  - Task assignment to team members
  - Due dates and labels
  - Comments and attachments
  - Activity log
  - Email notifications
  - Search functionality
- **Learning Outcomes:**
  - Drag-and-drop libraries (react-beautiful-dnd)
  - Complex state management (Redux)
  - Relational database design
  - File handling
  - Permission systems
  - Real-time updates
  - Advanced React patterns
- **Duration:** 4 weeks

## **Project 11: Social Media Dashboard**

**Tech Stack:** React, Node.js, MongoDB, Chart.js, OAuth

- **Features:**
  - User profiles
  - Create posts with images
  - Like and comment functionality
  - Follow/unfollow users
  - News feed algorithm
  - Notifications
  - Search users and posts
  - Analytics dashboard
- **Learning Outcomes:**
  - Social media architecture
  - Image optimization
  - Feed algorithms
  - OAuth authentication
  - Complex database queries
  - Caching strategies
  - Infinite scroll
  - CDN integration
- **Duration:** 4-5 weeks

## **Project 12: Job Board Platform**

**Tech Stack:** Next.js, PostgreSQL, Prisma, TailwindCSS

- **Features:**
  - Job listings with search and filters
  - Company profiles
  - User applications
  - Resume upload
  - Job alerts via email
  - Admin dashboard
  - Analytics for employers
  - Saved jobs functionality
- **Learning Outcomes:**
  - Next.js SSR and SSG
  - Prisma ORM
  - Advanced filtering
  - File handling (resumes)
  - Email automation
  - SEO optimization
  - Background jobs
  - Multi-role authentication
- **Duration:** 4 weeks

---

## **3 HARD PROJECTS (Advanced Features)**

## **Project 13: Video Streaming Platform (YouTube Clone)**

**Tech Stack:** MERN, AWS S3, FFmpeg, WebRTC

- **Features:**
  - Video upload with transcoding
  - Adaptive bitrate streaming (HLS)
  - Video player with controls
  - Like, comment, subscribe functionality
  - Channel management
  - Video recommendations algorithm
  - Live streaming capability
  - Search with autocomplete
  - View count and analytics
- **Learning Outcomes:**
  - Video processing and transcoding
  - Cloud storage (AWS S3)
  - CDN integration
  - Recommendation algorithms
  - Elastic search integration
  - WebRTC for live streaming
  - Microservices architecture
  - Caching (Redis)
  - Load balancing
- **Duration:** 6-8 weeks

## **Project 14: Collaborative Code Editor**

**Tech Stack:** React, Node.js, Socket.io, Monaco Editor, Docker

- **Features:**
  - Real-time collaborative editing
  - Multiple programming languages support
  - Code execution in sandbox
  - Syntax highlighting
  - Multiple cursors and selections
  - Chat functionality
  - File tree management
  - Version control integration
  - Share sessions via link
- **Learning Outcomes:**
  - Operational Transformation (OT)
  - Code execution in containers
  - Monaco Editor integration
  - WebSocket scaling
  - Docker containerization
  - Security in code execution
  - Conflict resolution

- Session management
- **Duration:** 6-7 weeks

## **Project 15: AI-Powered Learning Management System**

**Tech Stack:** MERN, TensorFlow.js, OpenAI API, Redis

- **Features:**
    - Course creation and management
    - Video lessons with progress tracking
    - AI-powered quiz generation
    - Personalized learning paths
    - Student analytics dashboard
    - Discussion forums
    - Certificate generation
    - Payment integration
    - AI chatbot for doubts
    - Plagiarism detection
  - **Learning Outcomes:**
    - AI/ML integration
    - Complex role-based access
    - Video streaming optimization
    - Analytics and reporting
    - Natural Language Processing
    - Certificate generation (PDF)
    - Forum architecture
    - Advanced caching
    - Performance optimization
  - **Duration:** 8 weeks
- 

## **2 RESEARCH-BASED PROJECTS (Cutting-Edge)**

### **Project 16: Decentralized Social Network (Blockchain)**

**Tech Stack:** React, Solidity, Ethereum, IPFS, Web3.js, Hardhat

- **Features:**
  - Wallet integration (MetaMask)
  - Create posts stored on blockchain
  - Decentralized file storage (IPFS)
  - Cryptocurrency tipping system
  - NFT profile pictures
  - DAO governance for platform decisions
  - Censorship-resistant content
  - Token-based incentives
  - Smart contract auditing
- **Learning Outcomes:**

- Blockchain fundamentals
- Smart contract development (Solidity)
- IPFS decentralized storage
- Web3 integration
- Cryptocurrency wallets
- Gas optimization
- Security in blockchain
- Testing smart contracts
- DApp architecture
- Ethereum development tools
- **Research Areas:**
  - Scalability solutions (Layer 2)
  - Consensus mechanisms
  - Decentralized identity
  - Token economics
- **Duration:** 10-12 weeks

## **Project 17: Real-Time Distributed System Monitor**

**Tech Stack:** Go/Rust, Kubernetes, Prometheus, Grafana, gRPC, Kafka

- **Features:**
  - Monitor multiple microservices
  - Real-time metrics collection
  - Distributed tracing
  - Log aggregation
  - Anomaly detection using ML
  - Auto-scaling based on metrics
  - Alert management
  - Service mesh integration
  - Performance profiling
  - Capacity planning tools
- **Learning Outcomes:**
  - Microservices architecture
  - Container orchestration (Kubernetes)
  - Observability patterns
  - Distributed systems concepts
  - Message queuing (Kafka)
  - gRPC communication
  - Time-series databases
  - Machine learning for anomalies
  - Infrastructure as Code
  - Cloud-native development
- **Research Areas:**
  - Edge computing
  - Service mesh patterns
  - Chaos engineering
  - Distributed consensus

- Performance optimization at scale
  - **Duration:** 12-14 weeks
- 

## 3 SYSTEM DESIGN (5-3-2 Distribution)

### ● 5 CORE/FUNDAMENTAL CONCEPTS

#### 1. Client-Server Architecture

##### What to Learn:

- Request-Response model
- HTTP/HTTPS protocols
- Stateless vs Stateful systems
- Client types (web, mobile, desktop)
- Server types (application, web, database)
- Communication patterns

##### Key Topics:

- DNS resolution process
- How browsers work
- REST principles
- API gateway concept
- Reverse proxy vs Forward proxy
- Session management

##### Practice Questions:

- Design a simple web application architecture
- Explain how a browser request reaches a server
- What happens when you type a URL in browser?

##### Resources:

- MDN Web Docs - HTTP
  - System Design Primer GitHub
- 

#### 2. Database Design Fundamentals

##### What to Learn:

- SQL vs NoSQL databases
- ACID properties
- Normalization (1NF, 2NF, 3NF)

- Indexing strategies
- Primary keys, Foreign keys
- Database transactions
- CAP theorem basics

**Key Topics:**

- When to use SQL vs NoSQL
- Schema design best practices
- Query optimization basics
- Data modeling
- Database sharding concepts
- Replication (Master-Slave)

**Practice Questions:**

- Design database schema for e-commerce site
- Explain ACID properties with examples
- When would you choose MongoDB over MySQL?

**Resources:**

- Database System Concepts book
  - MongoDB vs PostgreSQL comparison
- 

### 3. Caching Strategies

**What to Learn:**

- What is caching and why?
- Cache hit vs Cache miss
- Cache eviction policies (LRU, LFU, FIFO)
- Client-side vs Server-side caching
- CDN (Content Delivery Network)
- Browser caching

**Key Topics:**

- Redis and Memcached
- Cache-aside pattern
- Write-through vs Write-back
- Cache invalidation strategies
- TTL (Time To Live)
- Distributed caching

**Practice Questions:**

- How would you implement caching in a news website?



- Explain LRU cache and implement it
- When to invalidate cache?

**Resources:**

- Redis documentation
  - Caching strategies guide
- 

## **4. Load Balancing**

**What to Learn:**

- What is load balancing?
- Load balancer types (L4, L7)
- Load balancing algorithms (Round Robin, Least Connections, IP Hash)
- Horizontal vs Vertical scaling
- Health checks
- Sticky sessions

**Key Topics:**

- Nginx as load balancer
- AWS ELB, ALB, NLB
- Auto-scaling concepts
- Failover mechanisms
- Geographic load balancing
- Rate limiting

**Practice Questions:**

- Design load balancing for high-traffic website
- Round Robin vs Least Connections - when to use?
- How to handle server failures?

**Resources:**

- Nginx documentation
  - AWS load balancing guide
- 

## **5. API Design Principles**

**What to Learn:**

- RESTful API design
- HTTP methods (GET, POST, PUT, DELETE, PATCH)
- Status codes (200, 201, 400, 401, 404, 500)

- API versioning
- Authentication vs Authorization
- Rate limiting

**Key Topics:**

- REST vs GraphQL vs gRPC
- JWT tokens
- OAuth 2.0 flow
- API documentation (Swagger/OpenAPI)
- Pagination and filtering
- CORS (Cross-Origin Resource Sharing)

**Practice Questions:**

- Design REST API for Twitter
- How to version APIs?
- Explain JWT authentication flow

**Resources:**

- REST API tutorial
  - Swagger documentation
- 

## 3 MEDIUM/INTERMEDIATE CONCEPTS

### 6. Microservices Architecture

**What to Learn:**

- Monolith vs Microservices
- Service decomposition strategies
- Inter-service communication
- API Gateway pattern
- Service discovery
- Circuit breaker pattern

**Key Topics:**

- Docker and containers
- Kubernetes basics
- Message queues (RabbitMQ, Kafka)
- Event-driven architecture
- Saga pattern for distributed transactions
- Service mesh (Istio)

**Practice Questions:**

- When to use microservices vs monolith?
- Design microservices for e-commerce
- How to handle transactions across services?

**Deep Dive:**

- Netflix's microservices architecture
- 12-factor app methodology
- Domain-driven design

**Resources:**

- [Microservices.io](https://microservices.io)
  - Martin Fowler's articles
- 

## 7. Message Queues and Asynchronous Processing

**What to Learn:**

- Why asynchronous processing?
- Message queue concepts
- Publisher-Subscriber pattern
- Point-to-Point vs Pub-Sub
- Message acknowledgment
- Dead letter queues

**Key Topics:**

- RabbitMQ vs Kafka vs SQS
- Event sourcing
- CQRS pattern
- Guaranteed delivery
- Message ordering
- Idempotency

**Practice Questions:**

- Design notification system using message queue
- How Uber uses Kafka for real-time tracking?
- Handle message failures and retries

**Deep Dive:**

- At-least-once vs Exactly-once delivery
- Kafka internals
- Stream processing

**Resources:**

- RabbitMQ tutorials
  - Kafka documentation
- 

## 8. Data Partitioning and Sharding

### What to Learn:

- What is sharding?
- Horizontal vs Vertical partitioning
- Sharding strategies (Range, Hash, Geographic)
- Consistent hashing
- Rebalancing shards
- Hotspot problems

### Key Topics:

- Database partitioning
- Distributed databases
- Partition key selection
- Cross-shard queries
- MongoDB sharding
- Cassandra partitioning

### Practice Questions:

- Design sharding strategy for Instagram
- How to avoid hotspots?
- Consistent hashing explained

### Deep Dive:

- Distributed hash tables
- Virtual nodes concept
- Data rebalancing strategies

### Resources:

- MongoDB sharding guide
  - Consistent hashing paper
- 

## 2 ADVANCED CONCEPTS

## 9. Distributed Systems and Consensus

### What to Learn:

- CAP theorem in depth
- Eventual consistency
- Strong consistency
- Consensus algorithms (Paxos, Raft)
- Distributed transactions
- Two-phase commit

**Key Topics:**

- Leader election
- Distributed locks (Redlock)
- Vector clocks
- Conflict resolution
- Byzantine fault tolerance
- Quorum-based systems

**Practice Questions:**

- Explain CAP theorem with real examples
- Design distributed locking mechanism
- How does Raft consensus work?

**Deep Dive:**

- Google Spanner architecture
- Amazon Dynamo paper
- Blockchain consensus
- Distributed debugging

**Resources:**

- Designing Data-Intensive Applications book
  - MIT Distributed Systems course
- 

**10. Scalability and Performance Optimization****What to Learn:**

- Performance metrics (latency, throughput)
- Bottleneck identification
- Database optimization
- CDN strategies
- Async processing patterns
- Caching at multiple layers

**Key Topics:**

- Connection pooling

- Database query optimization
- N+1 query problem
- Lazy loading vs Eager loading
- HTTP/2 and HTTP/3
- WebSocket optimization
- Compression techniques
- Image optimization

#### **Practice Questions:**

- How to scale from 1000 to 1 million users?
- Optimize slow database queries
- Design high-performance API

#### **Deep Dive:**

- Netflix's scalability journey
- WhatsApp's architecture
- Profiling and monitoring tools
- Load testing strategies

#### **Resources:**

- High Scalability blog
  - System Design Interview book
- 

## **4 SYSTEM DESIGN TOOLS (4 Essential Tools)**

### **Tool 1: HLD (High-Level Design) Diagrams**

**Purpose:** Visualize system architecture at a high level

#### **What to Include:**

- Client applications (Web, Mobile)
- Load balancers
- Application servers
- Databases (SQL/NoSQL)
- Cache layers (Redis)
- Message queues (Kafka)
- CDN
- External APIs

#### **Tools to Use:**

- Draw.io (Diagrams.net)
- Lucidchart

- Excalidraw
- Miro
- Whimsical

### **Components to Master:**

- Boxes for services
- Arrows for data flow
- Cylinder for databases
- Cloud for external services
- Proper labeling

### **Practice:** Design HLD for:

1. URL Shortener (Bit.ly)
2. Instagram
3. Netflix
4. WhatsApp
5. Uber
6. Twitter
7. Amazon
8. YouTube
9. LinkedIn
10. Dropbox

### **Best Practices:**

- Keep it clean and simple
  - Use consistent symbols
  - Show data flow direction
  - Highlight critical paths
  - Include approximate numbers (if asked)
- 

## **Tool 2: API Contracts**

**Purpose:** Define clear interfaces between services

### **What to Include:**

- Endpoint URLs
- HTTP methods
- Request parameters
- Request body schema
- Response format
- Status codes
- Error responses
- Authentication headers

### Example Format:

Endpoint: POST /api/v1/users

Authentication: Bearer token required

Request Body:

```
{  
  "email": "user@example.com",  
  "password": "string (8-20 chars)",  
  "name": "string"  
}
```

Response 201:

```
{  
  "userId": "string",  
  "email": "string",  
  "createdAt": "timestamp"  
}
```

Errors:

400 - Invalid input

409 - Email already exists

### Tools to Use:

- Swagger/OpenAPI
- Postman collections
- API Blueprint
- RAML

**Practice:** Write API contracts for:

1. User authentication system
2. E-commerce cart
3. Social media posts
4. Payment processing
5. File upload service
6. Search functionality
7. Notification service
8. Chat messages
9. Booking system
10. Analytics tracking

### Best Practices:

- Version your APIs
- Use consistent naming
- Document all errors
- Include examples
- Specify data types



---

## Tool 3: Database Schema Design

**Purpose:** Structure data efficiently and logically

**What to Include:**

- Tables/Collections
- Columns/Fields with data types
- Primary keys
- Foreign keys
- Indexes
- Constraints
- Relationships (1:1, 1:N, N:N)

**Example Format:**

Table: Users

- id (UUID, Primary Key)
- email (VARCHAR(255), Unique, Indexed)
- password\_hash (VARCHAR(255))
- name (VARCHAR(100))
- created\_at (TIMESTAMP)
- updated\_at (TIMESTAMP)

Table: Posts

- id (UUID, Primary Key)
- user\_id (UUID, Foreign Key -> Users.id)
- content (TEXT)
- created\_at (TIMESTAMP)
- Indexes: user\_id, created\_at

**Tools to Use:**

- dbdiagram.io
- MySQL Workbench
- pgAdmin
- MongoDB Compass
- Draw.io

**Practice:** Design schemas for:

1. Twitter
2. Instagram
3. E-commerce site
4. LinkedIn
5. Airbnb

6. Uber
7. Stack Overflow
8. Reddit
9. Medium
10. Spotify

**Best Practices:**

- Normalize appropriately
  - Choose right data types
  - Index frequently queried fields
  - Plan for scalability
  - Document relationships
- 

**Tool 4: Bottleneck Analysis**

**Purpose:** Identify and resolve performance issues

**What to Analyze:****1. Database Bottlenecks:**

- Slow queries (N+1 problem)
- Missing indexes
- Lock contention
- Connection pool exhaustion
- Large table scans

**Solutions:**

- Add indexes
- Query optimization
- Read replicas
- Caching
- Connection pooling

**2. Network Bottlenecks:**

- High latency
- Bandwidth limitations
- Too many round trips
- Large payload sizes

**Solutions:**

- CDN usage
- Compression
- HTTP/2

- Data batching
- GraphQL for selective fields

### **3. Application Bottlenecks:**

- CPU-intensive operations
- Memory leaks
- Synchronous processing
- Inefficient algorithms

#### **Solutions:**

- Async processing
- Background jobs
- Algorithm optimization
- Horizontal scaling
- Code profiling

### **4. Infrastructure Bottlenecks:**

- Single point of failure
- Limited server resources
- No auto-scaling
- Poor load distribution

#### **Solutions:**

- Load balancers
- Auto-scaling groups
- Redundancy
- Health checks
- Monitoring and alerts

#### **Tools to Use:**

- New Relic
- DataDog
- AWS CloudWatch
- Grafana + Prometheus
- Chrome DevTools
- Database EXPLAIN plans

#### **Practice Questions:**

1. Website takes 5 seconds to load - analyze
2. API response time increased from 100ms to 2s
3. Database CPU at 95% - what to check?
4. Memory usage keeps increasing
5. Users reporting timeouts during peak hours

## Bottleneck Analysis Framework:

1. **Identify** - Monitor metrics
  2. **Measure** - Quantify the problem
  3. **Analyze** - Find root cause
  4. **Prioritize** - Impact vs Effort
  5. **Optimize** - Implement solution
  6. **Verify** - Measure improvement
- 

## 5 CORE CS FUNDAMENTALS

### OOPS (Object-Oriented Programming) - 4 Pillars

#### Pillar 1: Encapsulation

**Concept:** Bundling data and methods together, hiding internal details

#### Key Points:

- Private, public, protected access modifiers
- Getters and setters
- Data hiding
- Information hiding principle

#### Example (Java):

```
public class BankAccount {  
    private double balance; // Private data  
  
    public void deposit(double amount) {  
        if(amount > 0) {  
            balance += amount;  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

#### Interview Questions:

1. Why is encapsulation important?
2. Difference between public and private
3. Real-world examples of encapsulation

---

## Pillar 2: Abstraction

**Concept:** Hiding complex implementation, showing only essential features

### Key Points:

- Abstract classes
- Interfaces
- Hiding complexity
- Focus on "what" not "how"

### Example (Java):

```
abstract class Shape {  
    abstract double calculateArea();  
}  
  
class Circle extends Shape {  
    private double radius;  
  
    double calculateArea() {  
        return Math.PI * radius * radius;  
    }  
}
```

### Interview Questions:

1. Abstract class vs Interface
2. When to use abstraction?
3. Real-world abstraction examples

---

## Pillar 3: Inheritance

**Concept:** Creating new classes from existing ones, code reusability

### Key Points:

- IS-A relationship
- Parent-child hierarchy
- Method overriding
- Super keyword
- Types: Single, multilevel, hierarchical

### Example (Python):

```
class Vehicle:
    def __init__(self, brand):
        self.brand = brand

    def start(self):
        print("Vehicle starting")

class Car(Vehicle):
    def start(self):
        print(f"{self.brand} car starting")
```

### Interview Questions:

1. Types of inheritance
  2. Diamond problem
  3. Composition vs Inheritance
- 

### Pillar 4: Polymorphism

**Concept:** Same interface, different implementations

#### Key Points:

- Method overloading (compile-time)
- Method overriding (runtime)
- Dynamic binding
- Interfaces for polymorphism

#### Example (Java):

```
class Calculator {
    // Method overloading
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

// Method overriding
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
```

```
}  
  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Bark");  
    }  
}
```

### Interview Questions:

1. Overloading vs Overriding
  2. Runtime polymorphism example
  3. How polymorphism improves code?
- 

## OPERATING SYSTEM - 3 Topics (10 Questions Each)

### Topic 1: Process Management

#### 10 Questions:

1. **What is a process? Process vs Thread?**
  - Process: Program in execution with its own memory
  - Thread: Lightweight process sharing memory
  - Processes are isolated, threads share resources
2. **Process states and state diagram**
  - New, Ready, Running, Waiting, Terminated
  - State transitions explained
  - Context switching
3. **What is context switching? Overhead?**
  - Saving current process state
  - Loading next process state
  - Overhead: Time and resources
4. **Process scheduling algorithms**
  - FCFS, SJF, Round Robin, Priority
  - Preemptive vs Non-preemptive
  - Real-world usage
5. **What is a deadlock? Conditions?**
  - Mutual exclusion
  - Hold and wait
  - No preemption
  - Circular wait

## **6. Deadlock prevention vs avoidance vs detection**

- Prevention: Break one condition
- Avoidance: Banker's algorithm
- Detection: Resource allocation graph

## **7. Inter-process communication (IPC)**

- Shared memory
- Message passing
- Pipes, sockets
- Semaphores, mutex

## **8. Critical section problem**

- Race conditions
- Mutual exclusion solution
- Semaphores and monitors

## **9. Producer-Consumer problem**

- Bounded buffer
- Semaphore solution
- Real-world applications

## **10. Zombie and Orphan processes**

- Zombie: Completed but entry in table
- Orphan: Parent terminated
- How OS handles them

---

## **Topic 2: Memory Management**

### **10 Questions:**

#### **1. Virtual memory concept**

- Separation of logical and physical memory
- Allows larger programs than RAM
- Demand paging

#### **2. Paging vs Segmentation**

- Paging: Fixed-size blocks
- Segmentation: Variable-size blocks
- Pros and cons of each

#### **3. Page replacement algorithms**

- FIFO, LRU, Optimal
- Belady's anomaly
- Working set model



#### **4. Thrashing - what and how to prevent?**

- Excessive paging activity
- Causes: Too many processes
- Solutions: Reduce multiprogramming degree

#### **5. Memory allocation techniques**

- Contiguous: First-fit, Best-fit, Worst-fit
- Non-contiguous: Paging
- Fragmentation issues

#### **6. Translation Lookaside Buffer (TLB)**

- Fast page table cache
- TLB hit vs miss
- Performance improvement

#### **7. Stack vs Heap memory**

- Stack: Local variables, function calls
- Heap: Dynamic allocation
- Memory leaks in heap

#### **8. Internal vs External fragmentation**

- Internal: Wasted space within allocated block
- External: Free space scattered
- Solutions for each

#### **9. Demand paging**

- Load pages only when needed
- Page fault handling
- Lazy loading concept

#### **10. Memory management in modern OS**

- Linux memory management
- Windows virtual memory
- Memory compression

---

### **Topic 3: File Systems and I/O**

#### **10 Questions:**

##### **1. File system structure**

- Boot block, superblock
- Inode table
- Data blocks
- Directory structure

## **2. File allocation methods**

- Contiguous allocation
- Linked allocation
- Indexed allocation
- Pros and cons

## **3. Disk scheduling algorithms**

- FCFS, SSTF, SCAN, C-SCAN
- LOOK, C-LOOK
- Seek time optimization

## **4. RAID levels**

- RAID 0: Striping
- RAID 1: Mirroring
- RAID 5: Parity
- RAID 10: Combination

## **5. Inode structure**

- File metadata storage
- Direct and indirect blocks
- How file data is accessed

## **6. Hard link vs Soft link**

- Hard link: Same inode
- Soft link: Pointer to file
- Differences and use cases

## **7. File permissions in Unix/Linux**

- Read, write, execute
- User, group, others
- chmod command

## **8. Buffering and caching in I/O**

- Why buffering?
- Write-back vs Write-through cache
- Page cache in Linux

## **9. Journaling file system**

- Maintains log of changes
- Crash recovery
- ext4, NTFS

## **10. SSD vs HDD**

- SSD: No moving parts, faster
- HDD: Mechanical, cheaper
- File system considerations

---

## COMPUTER NETWORKS - 3 Topics (8 Questions Each)

### Topic 1: Network Layers and Protocols

#### 8 Questions:

1. **OSI model - 7 layers explained**

- Physical, Data Link, Network, Transport
- Session, Presentation, Application
- Functions of each layer

2. **TCP vs UDP**

- TCP: Connection-oriented, reliable
- UDP: Connectionless, fast
- Use cases for each

3. **Three-way handshake (TCP)**

- SYN → SYN-ACK → ACK
- Connection establishment
- Why three steps?

4. **HTTP vs HTTPS**

- HTTPS = HTTP + SSL/TLS
- Encryption and security
- Certificate verification

5. **DNS - How it works?**

- Domain name to IP resolution
- DNS hierarchy
- Recursive vs Iterative queries

6. **HTTP methods (GET, POST, PUT, DELETE)**

- Idempotent methods
- Safe methods
- When to use each

7. **Status codes (200, 404, 500, etc.)**

- 2xx: Success
- 4xx: Client errors
- 5xx: Server errors

8. **IPv4 vs IPv6**

- Address exhaustion in IPv4
- IPv6 address format
- Transition mechanisms

---

## Topic 2: Network Security and Performance

### 8 Questions:

#### 1. How does HTTPS work? SSL/TLS handshake

- Client Hello
- Server certificate
- Key exchange
- Encrypted communication

#### 2. What is a firewall?

- Packet filtering
- Stateful inspection
- Application-level gateway

#### 3. DDoS attack - what and how to prevent?

- Distributed denial of service
- Types: SYN flood, UDP flood
- Mitigation: Rate limiting, CDN

#### 4. Load balancing strategies

- Round robin, Least connections
- Layer 4 vs Layer 7
- Health checks

#### 5. CDN - How it improves performance?

- Content caching at edge
- Reduced latency
- DDoS protection

#### 6. Cookies vs Sessions vs Tokens

- Cookies: Client-side storage
- Sessions: Server-side storage
- JWT tokens: Stateless auth

#### 7. CORS - Why it exists?

- Cross-Origin Resource Sharing
- Same-origin policy
- Preflight requests

#### 8. Latency vs Throughput vs Bandwidth

- Latency: Time delay
- Throughput: Data transfer rate
- Bandwidth: Maximum capacity

---

## Topic 3: Application Layer Concepts

### 8 Questions:

#### 1. REST API principles

- Stateless
- Resource-based URLs
- HTTP methods
- Cacheable

#### 2. WebSockets - when to use?

- Full-duplex communication
- Real-time applications
- vs HTTP polling

#### 3. GraphQL vs REST

- GraphQL: Query what you need
- REST: Fixed endpoints
- Trade-offs

#### 4. Rate limiting - implementation

- Token bucket
- Leaky bucket
- Fixed window, Sliding window

#### 5. API versioning strategies

- URL versioning (/v1/users)
- Header versioning
- Backward compatibility

#### 6. OAuth 2.0 flow

- Authorization code grant
- Access tokens
- Refresh tokens

#### 7. Microservices communication

- Synchronous: REST, gRPC
- Asynchronous: Message queues
- Service discovery

#### 8. What happens when you type a URL?

- DNS resolution
- TCP connection
- HTTP request
- Server processing

- Response rendering
- 

## **DATABASE MANAGEMENT SYSTEMS (DBMS) - 3 Topics (6 Questions Each)**

### **Topic 1: Database Design and Normalization**

#### **6 Questions:**

##### **1. Normalization - 1NF, 2NF, 3NF, BCNF**

- 1NF: Atomic values
- 2NF: No partial dependency
- 3NF: No transitive dependency
- BCNF: Every determinant is key

##### **2. ER Diagram components**

- Entities and attributes
- Relationships (1:1, 1:N, N:N)
- Cardinality and participation

##### **3. Primary key vs Foreign key vs Unique key**

- Primary: Unique identifier
- Foreign: References another table
- Unique: Can be null, unique values

##### **4. Indexing - types and when to use**

- B-Tree, Hash, Bitmap indexes
- Clustered vs Non-clustered
- Index on frequently queried columns

##### **5. SQL vs NoSQL - when to use?**

- SQL: Structured, ACID, relationships
- NoSQL: Flexible schema, scalability
- Use cases for each

##### **6. Denormalization - why and when?**

- Improve read performance
  - Reduce joins
  - Trade-off: Data redundancy
- 

### **Topic 2: Transactions and Concurrency**

#### **6 Questions:**

## 1. ACID properties explained

- Atomicity: All or nothing
- Consistency: Valid state
- Isolation: Concurrent transactions
- Durability: Permanent changes

## 2. Transaction isolation levels

- Read Uncommitted
- Read Committed
- Repeatable Read
- Serializable
- Dirty reads, phantom reads

## 3. Locking mechanisms

- Shared locks (read)
- Exclusive locks (write)
- Deadlock prevention

## 4. Optimistic vs Pessimistic locking

- Optimistic: Version checking
- Pessimistic: Lock before access
- When to use each

## 5. Two-phase commit protocol

- Prepare phase
- Commit phase
- Distributed transactions

## 6. Database deadlocks - detection and prevention

- Wait-for graph
- Timeout approach
- Ordering resources

---

## Topic 3: Query Optimization and Performance

### 6 Questions:

#### 1. Query optimization techniques

- Use indexes
- Avoid SELECT \*
- Limit result set
- Use JOIN instead of subqueries

#### 2. EXPLAIN plan - how to read?

- Execution order
  - Index usage
  - Full table scan detection
  - Cost analysis
  - 3. **N+1 query problem**
    - Multiple queries in loop
    - Solution: JOIN or eager loading
    - ORMs and lazy loading
  - 4. **Database sharding**
    - Horizontal partitioning
    - Shard key selection
    - Challenges: Cross-shard queries
  - 5. **Replication - Master-Slave**
    - Master: Write operations
    - Slave: Read replicas
    - Eventual consistency
  - 6. **Connection pooling**
    - Reuse database connections
    - Reduces overhead
    - Configuration parameters
- 

## **6 PROGRAMMING LANGUAGES**

### **Language 1: JavaScript (Development Language)**

#### **Why JavaScript?**

- Frontend (React, Vue, Angular)
- Backend (Node.js)
- Full-stack development
- Largest ecosystem (npm)

#### **Core Topics to Master:**

##### **1. Fundamentals**

- Variables (var, let, const)
- Data types and type coercion
- Operators and expressions
- Control flow (if, switch, loops)
- Functions (regular, arrow, IIFE)
- Scope (global, function, block)



- Hoisting

## **2. Advanced Concepts**

- Closures
- Callbacks
- Promises
- Async/Await
- Event loop
- Prototypes and inheritance
- this keyword
- call, apply, bind

## **3. ES6+ Features**

- Destructuring
- Spread/Rest operators
- Template literals
- Default parameters
- Classes
- Modules (import/export)
- Map, Set, WeakMap, WeakSet
- Symbols
- Generators
- Iterators

## **4. DOM Manipulation**

- Selecting elements
- Event handling
- Creating and modifying elements
- Event delegation
- LocalStorage/SessionStorage

## **5. Modern JavaScript**

- Array methods (map, filter, reduce, forEach)
- String methods
- Object methods
- JSON handling
- Fetch API
- Error handling (try/catch)

## **Practice Resources:**

- JavaScript.info
- MDN Web Docs
- Eloquent JavaScript book
- FreeCodeCamp challenges

---

## Language 2: Python (Core Language 1)

### Why Python?

- Data structures and algorithms
- Scripting and automation
- Backend development (Django, Flask)
- Data science and ML

### Core Topics to Master:

#### 1. Fundamentals

- Variables and data types
- Operators
- Control flow
- Functions
- Modules and packages
- File I/O
- Exception handling

#### 2. Data Structures

- Lists, Tuples, Sets, Dictionaries
- List comprehensions
- Dictionary comprehensions
- Collections module (Counter, defaultdict)
- heapq, bisect modules

#### 3. OOP in Python

- Classes and objects
- Inheritance
- Polymorphism
- Encapsulation
- Magic methods (**init**, **str**)
- Property decorators
- Class and static methods

#### 4. Advanced Features

- Decorators
- Generators
- Context managers (with statement)
- Lambda functions
- map, filter, reduce
- \*args and \*\*kwargs

- List slicing

## **5. Standard Library**

- os, sys modules
- datetime
- re (regex)
- json
- itertools
- functools

### **Practice Resources:**

- Python.org documentation
  - LeetCode Python
  - Real Python tutorials
  - Automate the Boring Stuff book
- 

## **Language 3: Java (Core Language 2)**

### **Why Java?**

- Enterprise applications
- Android development
- Strong typing and OOP
- Performance and scalability

### **Core Topics to Master:**

#### **1. Fundamentals**

- Data types (primitive and reference)
- Operators
- Control statements
- Arrays
- Strings and String methods
- Input/Output

#### **2. OOP in Java**

- Classes and objects
- Constructors
- Inheritance
- Polymorphism
- Abstraction (abstract classes, interfaces)
- Encapsulation
- Access modifiers
- Static keyword

- Final keyword

### 3. Collections Framework

- List (ArrayList, LinkedList)
- Set (HashSet, TreeSet, LinkedHashSet)
- Map (HashMap, TreeMap, LinkedHashMap)
- Queue (PriorityQueue)
- Iterators
- Comparable vs Comparator

### 4. Advanced Concepts

- Exception handling (try-catch-finally)
- Multithreading
- Synchronization
- Generics
- Lambda expressions (Java 8+)
- Stream API
- Functional interfaces

### 5. Important Topics

- File handling
- Serialization
- JDBC basics
- Packages and imports
- Wrapper classes
- Autoboxing/Unboxing

### Practice Resources:

- Oracle Java documentation
- GeeksforGeeks Java
- Head First Java book
- HackerRank Java challenges



## STUDY PLAN TIMELINE

### Month 1-3: DSA Foundation (Patterns 1-25)

- **Week 1-4:** Arrays and Strings (Patterns 1-10)
- **Week 5-8:** Linked Lists and Stacks (Patterns 11-20)
- **Week 9-12:** Trees and Graphs (Patterns 21-25)
- **Daily:** 3-4 questions
- **Projects:** Start Project 1-3

## **Month 4-6: DSA Intermediate + Projects**

- **Week 13-18:** Recursion and Backtracking (Patterns 31-40)
- **Week 19-24:** Dynamic